

The Database Schema

Job Application Tracker — what lives in the database and why

What is a schema?

A schema is the blueprint for your database. It defines what tables exist, what columns they have, what type of data each column holds, and what rules are enforced automatically. Any data that doesn't follow these rules gets rejected before it's ever saved.

Our schema lives in `backend/schema.sql`. Every developer runs this file once during local setup.

Our table: applications

One table for the MVP. Every job you track is one row. Here's every column:

Column	Type	Required?	What it stores
<code>id</code>	UUID	Auto	Unique ID for every row. Generated automatically — never set this yourself.
<code>company_name</code>	TEXT	Yes	The company you applied to.
<code>job_title</code>	TEXT	Yes	The role title.
<code>status</code>	ENUM	Default: draft	Where the application stands. Must be one of the allowed values — see below.
<code>job_url</code>	TEXT	No	Link to the original job posting.
<code>date_applied</code>	DATE	No	Date the application was submitted.
<code>notes</code>	TEXT	No	Freeform. Recruiter name, salary, interview prep — anything.
<code>created_at</code>	TIMESTAMP	Auto	Set when the row is created. Never changes.
<code>updated_at</code>	TIMESTAMP	Auto	Updates automatically every time the row changes.

The status field — allowed values only

Status is an ENUM. The database hard-rejects anything not on this list — you can't accidentally set it to "pending" or "maybe".

Value	Meaning
draft	Default. Created but not yet submitted.
applied	Application has been submitted.
interviewing	Interview scheduled or in progress.
offered	You received an offer.
rejected	Application was not successful.
withdrawn	You pulled yourself out of the process.

What happens automatically

Three things run without any code from you:

Thing	What it does
UUID generation	pgcrypto generates a unique id for every new row. Never pass an id when creating a record.
created_at	Set once on insert. Never touched after that.
updated_at trigger	A database trigger fires on every UPDATE and stamps the current time. The app doesn't do anything — the DB handles it.

Indexes

An index is like a table of contents — without one the DB scans every row. With one it jumps straight to the right data. We have two:

Index	Why it exists
idx_applications_status	Makes filtering by status fast.
idx_applications_created_at	Our default sort is newest first. This makes ORDER BY created_at DESC instant.

Running the schema

Run once during local setup. This wipes and recreates everything — existing local data will be deleted.

```
docker exec -i job-tracker-db psql -U postgres -d job_tracker < backend/schema.sql
```

Heads up: The schema uses DROP TABLE IF EXISTS at the top. Running it again deletes all local data. Fine for development — never run this against production.